
MLPC Report - Task 4: Data Classification

Team Temporal Frameworks

Sebastian Pichler

Laurin Siebert

1 Dataset Preparation

1.1 Label Aggregation

To derive class labels for every timestep, we stuck to our approach already used in the previous task. We use a weighted majority vote, where the annotations from the creator of the recording are slightly stronger, assuming the creator might have more precise information about the events. The decision boundary for binarisation is set to 50%. Given some unclear areas and only 2 annotators, the creator will always be the deciding factor. Given more than 2 annotators, the majority can outvote the creator.

However, as we already saw during the annotation verification phase of Task 3, the labels in this version of the dataset often heavily disagree. It would have been great if we were granted access to the verified labels, which would most certainly have led to better classification results in this task.

1.2 Data Split

The goal is to split the data in a way that prevents information leakage while maintaining a consistent class distribution across splits.

One source of information leakage in this task arises from correlations between samples. In particular, consecutive time frames within a recording are highly correlated, and recordings produced by the same collector often share similar acoustic environments. To avoid learning bias all timeframes from the same sample, as well as all recordings from the same collector, must be assigned exclusively to a single split.

We initially considered a per-class `StratifiedGroupKFold` approach, which achieves both creator separation and well-preserved class distributions for each class independently. However, this approach results in a different train/test split for each class, making it impossible to evaluate all classifiers on a common test set. Since a shared test distribution is required for consistent evaluation, we instead adopt a single global split.

To this end, we perform a group-based split using the collector ID as grouping variable. Collectors are divided into training and test groups using an 80/20 split, ensuring that no collector appears in both sets. Due to the varying number of recordings per collector, this results in an approximate rather than exact sample-level split (2584 training and 1072 test recordings in our case).

To preserve class distributions under this grouping constraint, we evaluate multiple random seeds (1–100) and select the split that minimizes the maximum per-class deviation from the overall class distribution. The selected split (seed 6) yields 286 training collectors and 72 test collectors, with a maximum deviation of 3.4 percentage points across classes, which we consider acceptable given the strict grouping constraint.

Maintaining similar class distributions across training and test sets is important to ensure that the model is trained and evaluated under comparable conditions.

Table 1 shows that the class distributions are well preserved across the splits, confirming that the chosen procedure achieves a good balance between leakage prevention and stratification.

Class	Overall	Train	Test
bell_ringing	7.36%	7.08%	8.02%
coffee_machine	5.55%	5.69%	5.22%
cutlery_dishes	20.98%	21.94%	18.66%
door_open_close	29.98%	29.95%	30.04%
footsteps	47.16%	47.83%	45.52%
keyboard_typing	19.61%	19.66%	19.50%
keychain	18.24%	18.42%	17.82%
light_switch	7.47%	6.89%	8.86%
microwave	10.50%	11.03%	9.24%
phone_ringing	20.62%	20.51%	20.90%
running_water	26.86%	27.05%	26.40%
toilet_flushing	10.97%	11.11%	10.63%
vacuum_cleaner	8.78%	8.55%	9.33%
wardrobe_drawer_open_close	14.82%	15.02%	14.37%
window_open_close	12.50%	12.35%	12.87%

Table 1: Class distribution across dataset splits

1.3 Preprocessing

We concatenate all acoustic features into a single feature matrix of shape (N_frames, 960). From these, we select a subset of features based on correlation analysis performed during the data exploration phase: ZCR, MFCC and its first and second derivatives, spectral contrast and signal power (all four statistics each), resulting in 420 dimensions. We exclude the mel spectrogram (512 dimensions) as it dominates the feature space without providing proportional benefit for classification. All selected features are standardised using z-score normalisation (zero mean, unit variance), with the scaler fitted exclusively on the training data and applied to validation and test sets to prevent data leakage.

2 Evaluation

2.1 Evaluation Criterion

We use macro-averaged F1 score as our evaluation criterion, as it treats all 15 classes equally regardless of their frequency and penalises both false positives and false negatives. This is especially important given the strong class imbalance we already observed in Task 3 and in Table 1. Macro-averaging computes the F1 score separately for each class and takes the arithmetic mean, resulting in equal weight distribution across all classes. In contrast, micro-averaged F1 aggregates predictions globally and is dominated by frequent classes, while weighted F1 would offer a middle ground by weighting each class by its support.

$$F1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (1)$$

$$\text{Macro F1} = \frac{1}{C} \sum_{i=1}^C F1_i \quad (2)$$

2.2 Baseline Performance

As expected, the baseline classifier always predicts `negative` for every timestep, so it never produces a true positive for any sound event. This makes both precision and recall for the positive class zero and therefore $F1 = 0$ and $\text{Macro F1} = 0$.

3 Experiments

3.1 Hyperparameters

The first model classifier we chose is an XGBoost Classifier. While XGBoost is from the family of ensemble tree-based classifiers it trains trees sequentially based on gradient boosting, unlike a Random Forest model would do. In XGBoost misclassified samples are given a weight and thus more focus for the next iteration. Hyperparameters we tuned included:

- "n_estimators": The number of trees to train. Too few trees can lead to underfitting, while too many may increase training time and risk overfitting.
- "learning_rate": The learning rate, i.e. step size. Smaller learning rates generally improve generalisation but require more trees to converge and thus also more time.
- "max_depth": The maximum depth of each decision tree. Larger depths allow more complex models but might lead to overfitting.
- "min_child_weight": The minimum amount of data required to make a new split.
- "subsample": The portion of training data used for each tree. Values below 1 introduce randomness into the model's training.
- "scale_pos_weight": A weighting factor that increases a target's weight. Might help in case of imbalanced data.

Choosing the best values for these was done via grid search over different combinations.

As a second classifier, we implemented a multi-label neural network using PyTorch. The model consists of a shared backbone with three fully-connected layers ($420 \rightarrow 256 \rightarrow 128 \rightarrow 64$), each followed by batch normalisation, ReLU activation and dropout (except for the final layer), which learns a common audio representation across all classes. On top of this, 15 independent binary output heads ($64 \rightarrow 1$) predict the presence of each single class. Since the dataset is heavily imbalanced at the frame level, we replaced the standard binary cross-entropy loss with focal loss, which down-weights easy negative samples dynamically based on the prediction confidence and additionally up-weights the positive class via a fixed α parameter. The model is trained with the Adam optimizer and a cosine annealing learning rate schedule, decaying from the starting value to near zero over the training epochs. We tried optimizing the following hyperparameters:

- "alpha": The positive class weight in the focal loss. Higher values assign more importance to positive (present) frames, which helps with heavily imbalanced classes such as `light_switch`.
- "gamma": Higher values suppress the gradient contribution of easy negative examples in the focal loss more aggressively, forcing the model to focus on ambiguous frames.
- "dropout": The dropout rate applied after the first two backbone layers. Higher values reduce overfitting by randomly deactivating neurons during training.
- "learning_rate": The initial learning rate for the Adam optimiser, decayed to near zero via cosine annealing over the course of training. Smaller values lead to more careful weight updates but slower convergence.
- "weight_decay": The L2 regularisation coefficient in the Adam optimiser. Penalises large weights and reduces overfitting.
- "epochs": The number of full passes through the training data. Too few leads to underfitting; too many risks overfitting, mitigated here by the learning rate schedule.
- "threshold": The decision threshold applied per class to convert predicted probabilities into binary labels. Tuning this per class is particularly important for rare sound events where the default of 0.5 is too conservative.

3.2 Classifier Comparison

Both evaluated classifiers substantially outperform the baseline classifier introduced in Section 2.2. While the baseline always predicts the negative class and therefore achieves a Macro F1 score of 0, both learned models are able to detect meaningful acoustic event patterns across most classes.

Overall, the neural network trained with focal loss achieves slightly higher Macro F1 performance than the XGBoost classifier. Improvements are particularly visible for medium-frequency classes such as `coffee_machine`, `toilet_flushing`, and `cutlery_dishes`. This suggests that the neural network generalises better on standard cases.

Class	XGBoost F1	Focal-Loss NN F1
bell_ringing	0.431	0.521
coffee_machine	0.416	0.529
cutlery_dishes	0.502	0.579
door_open_close	0.309	0.358
footsteps	0.475	0.510
keyboard_typing	0.677	0.704
keychain	0.550	0.593
light_switch	0.185	0.135
microwave	0.550	0.599
phone_ringing	0.627	0.651
running_water	0.757	0.761
toilet_flushing	0.506	0.611
vacuum_cleaner	0.705	0.676
wardrobe_drawer_open_close	0.221	0.234
window_open_close	0.228	0.200
Macro Avg F1	0.476	0.511

Table 2: Per-class F1-score comparison between XGBoost and the Focal-Loss Neural Network.

The XGBoost classifier nevertheless performs competitively and achieves strong results on several structured acoustic events such as `running_water`, `vacuum_cleaner`, and `keyboard_typing`. It performs better on underrepresented labels like `light_switch` and `window_open_close` because of the weighting strategy via "scale_pos_weight" giving those labels more emphasis. Tree-based ensemble methods are generally robust on tabular feature representations and require comparatively little preprocessing. In addition, XGBoost is less computationally expensive to train and easier to interpret than neural network models.

Both models still struggle on acoustically similar or underrepresented classes such as `light_switch`, `window_open_close`, and `wardrobe_drawer_open_close`. These events often contain short transient sounds with overlapping frequency characteristics, making them difficult to distinguish.

In summary, both classifiers clearly improve upon the baseline and demonstrate that the selected acoustic features contain meaningful information for the classification task. The neural network with focal loss achieves the strongest overall performance and appears better suited for handling class imbalance and complex feature interactions, whereas XGBoost offers a robust and computationally efficient alternative with competitive results on many classes.

4 Case Study and Reflection

4.1 Analysis on example recordings

After analysing the behaviour of the XGBoost model on selected samples from the test set, we observe a strong tendency for the model to overpredict the label `microwave`. In the shown example in Fig. 1 we find the frame-wise activations to match the ground truth reasonably well. The true class label `phone_ringing` is correctly classified.

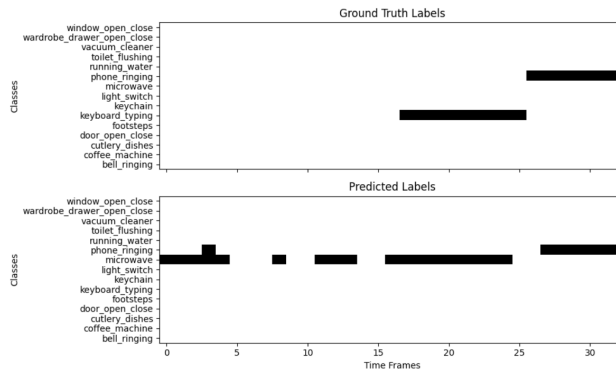


Figure 1: The model predicts microwave but it recognises the pattern of activation.

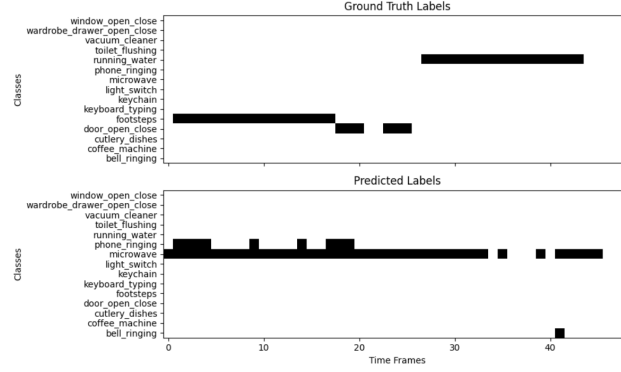


Figure 2: Once more microwave dominates even though the class is not present in the file.

Fig.2 in contrast shows a case where microwave is predicted throughout most of the recording despite the class not being present. This shows the model's tendency to assign this label as a general strategy, which leads to the high number of false positives in the confusion matrix Fig. 3. Overall, these examples give an idea of the strong bias towards the class microwave.

4.2 Classification results

The XGBoost model performs competitively overall and clearly better than the baseline. In a direct comparison its biggest strength over another classifier like the ANN lies in predicting the underrepresented labels. It outperforms many other classic machine learning algorithms while falling short overall to modern neural networks. We can imagine a model that combines a CNN's strength for structure and an RNN's strength of using the sequential nature of the data to perform even better. A transformer model would also be able to use the underlying structure of audio data better through a self-attention mechanism. The biggest weakness of the trained XGBoost model is the overprediction of the class microwave. The confusion matrix Fig. 3 on the test set shows that this label was predicted incorrectly much more often than expected.

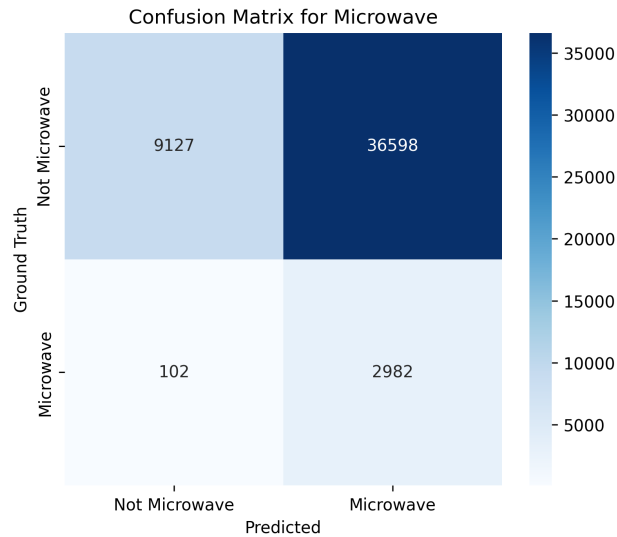


Figure 3: The confusion matrix for class label microwave.

5 Disclosure of LLM and AI Tool Use

For this task ChatGPT-5.3, Gemini-3.1 and Claude Sonnet 4.6 were used to brainstorm as well as to debug code quickly and perform minor spell checking. The correctness was verified by usage of own knowledge. Suggestions are never trusted fully but rather taken as a possibility to be considered.